



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Domain-Specific Language for Polynomial Universal Hash Functions

Master Thesis

Michael Mörschell

June 26, 2026

Advisors: Prof. Dr. Markus Püschel, Tommaso Pegolotti
Department of Computer Science, ETH Zürich

Acknowledgments

You can use this section to thank whoever you want to in the context of your thesis. This can of course include family and friends, colleagues, your advisor, and in general anyone that provide some form of support. You can also take the opportunity here to give credit to people who may have enlightened you with discussions, and also certainly your co-authors of an academic paper related to the thesis, should there be one.

Abstract

In this document we show you how to structure your paper and give some guidelines regarding your (Math and English) writing.

Contents

Contents	iii
1 Introduction	1
1.1 Features	1
1.1.1 Extra package includes	2
1.1.2 Layout setup	2
1.1.3 Theorem setup	2
1.1.4 Macro setup	3
2 Writing scientific texts in English	5
2.1 Basic writing rules	5
2.2 Being nice to the reader	5
2.3 A few important grammar rules	6
2.4 Things you (usually) don't say in English	9
3 Typography	10
3.1 Punctuation	10
3.2 Spacing	11
3.3 Choice of 'fonts'	12
3.4 Displayed equations	13
3.5 Floats	14
4 More Tips on Math/L^AT_EX	15
4.1 The basics	15
4.2 Some common math for reference/uniformity	16
4.3 Figures and Algorithms	16
4.4 Other tips and tricks	17
5 How a Typical Academic Paper is Structured	19
5.1 Preamble	19
5.2 Introduction	19

5.3	Notation and Preliminaries	20
5.4	Main Content (Approach/Analysis)	20
5.4.1	Algorithm or Method Description	20
5.4.2	Theoretical Analysis	21
5.4.3	Experimental Results (If Applicable)	21
5.5	Conclusion and Future Work	21
5.6	References	21
5.7	Appendices (If Needed)	22
6	Introduction	23
7	Notation and Preliminaries	25
8	Algorithm and Analysis	28
9	Conclusion	34
	Bibliography	36
	Dummy Appendix	37

Chapter 1

Introduction

This is version v2.0 of the template.

We assume that you found this template on our institute's website, so we do not repeat everything stated there (although some things are repeated for your convenience as far as $\text{\LaTeX}$ code is concerned). Consult the website again for pointers to further reading about $\text{\LaTeX}$. This chapter only gives a brief overview of the files you are looking at. Before we start, we would like to point out that once you read all the information, you can switch the flag `studentwriting` (found in `thesis.tex`) to true so that you start with a cleaner template (basically starting from Chapter 6) in which you can fill in your thesis content.

1.1 Features

The rest of this document shows off a few features of the template files. Look at the source code to see which macros we used!

The template is divided into $\text{\TeX}$ files as follows:

1. `thesis.tex` is the main file.
2. `extrapackages.tex` holds extra package includes.
3. `layoutsetup.tex` defines the style used in this document.
4. `theoremsetup.tex` declares the theorem-like environments.
5. `macrosetup.tex` defines extra macros that you may find useful.
6. `introduction.tex` contains this text.
7. `sections.tex` is a quick demo of each sectioning level available (not included in the pdf, but you can consult it).

8. `refs.bib` is an example bibliography file. You can use BibTeX to quote references. For example, read [2] if you can get a hold of it.

1.1.1 Extra package includes

The file `extrapackages.tex` lists some packages that usually come in handy. Simply have a look at the source code. We have added the following comments based on our experiences:

REC This package is recommended.

OPT This package is optional. It usually solves a specific problem in a clever way.

ADV This package is for the advanced user, but solves a problem frequent enough that we mention it. Consult the package's documentation.

As a small example, here is a reference to the Section *Features* typeset with the recommended *cleverref* package. As the name implies, this package cleverly identifies what the thing it is referring to is (Section, Theorem, Remark, etc.). Note that we have not typed the word 'Section' in the L^AT_EX code. See the code after `\usepackage{cleverref}` in the `thesis.tex` file for details.

See Section 1.1.

1.1.2 Layout setup

This defines the overall look of the document – for example, it changes the chapter and section heading appearance. We consider this a 'do not touch' area. Take a look at the excellent *Memoir documentation* before changing it.

1.1.3 Theorem setup

This file defines a bunch of theorem-like environments.

Lemma 1.1 *An example lemma.*

Proof Proof text goes here. □

Note that the q.e.d. symbol moves to the correct place automatically (because of `ntheorem`) if you end the proof with an `enumerate` or `displaymath`. You do not need to use `\qedhere` as with *amsthm*.

Theorem 1.2 (Some Famous Guy) *An example theorem.*

Proof This proof

1. ends in an enumerate. □

Proposition 1.3 *Note that all theorem-like environments are by default numbered on the same counter, i.e. we have Theorem 1.1 and then Theorem 1.2.*

Proof This proof ends in a display like so:

$$f(x) = x^2. \quad \square$$

1.1.4 Macro setup

In the macro setup file we show how to define some basic macros, and how to use a neat feature of the *mathtools* package:

$$|a|, \quad \left| \frac{a}{b} \right|, \quad \left| \frac{a}{b} \right|.$$

What is happening above is that whatever goes into the `\abs` command gets surrounded with vertical bars. If you add a `*` to the command, it sizes the bars automatically to cover the whole expression. Or you can pass an argument to set the size yourself (i.e. `\big`, `\bigg`, `\Big`, etc.), which is a good approach in many cases since the automatic version is not always the best. We have also given macros for Landau notation.

$$\begin{aligned} \mathcal{O}(n^2), \quad \mathcal{O}(n^{\frac{1}{2}}), \quad \mathcal{O}\left(n^{\frac{1}{2}}\right) \\ \Omega(n^3), \quad \Omega\left(n^{e^4}\right), \quad \Omega(\log n) \\ \Theta(n^4), \quad o\left(n^2 + n^{\frac{5n}{\log n}}\right), \quad \omega\left(\log^2 n\right) \end{aligned}$$

When in doubt, use the starred version and then check to see if things look weird. In such cases, experiment with fixed sizes. After a while, you will be able to guess the correct size reliably. An example where it is better to use your own judgement (and which shows the differences between automatic sizing with `\left(...\right)` and manual with `\biggl(...\biggr)` under the hood in our macros):

$$\left(\sum_{i=1}^n\right)^2.$$

Compare to the manually-sized:

$$\left(\sum_{i=1}^n\right)^2.$$

An exception to the rule of using the starred version is when writing inline math, where we don't want to use unnecessarily large vertical spacing,

because that messes with the line spacing. For more details, see Rule 4.13. Some more macros (it is also not hard to define your own if you need something extra):

$$\mathbb{P}[v \text{ has degree } 5], \quad \mathbb{E}[X], \quad \max\{Y, Z\} \\ \min\{X + Z, Y + W\}, \quad \exp(\log n), \quad \log\left(x^{n^d}\right), \quad \text{Var}[Z]$$

We also define an example macro for when one needs to regularly refer to a word with a different font, like the `ONEMAX` problem. The use of the `\xspace` command is important here, as it allows for correct spacing after the macro, regardless of whether it is followed by punctuation or not, both of which can happen when discussing `ONEMAX`.

Another thing we do in this file is to define `\eps` to be ε because this style is more common and nicer to look at (compare ε to ϵ). We also redefine `\phi` to be `\varphi` (φ). You can still use the “old” versions with `\epsilon` and `\oldphi` (ϕ). There are other characters that have variations for example, ρ versus ϱ . Now that we mention characters, a common mistake is to use l instead of ℓ in math, try to avoid it.

We also define some more macros, which are explained in Chapter 4.

Chapter 2

Writing scientific texts in English

This chapter was originally a separate document written by Reto Spöhel. It is reprinted here so that the template can serve as a quick guide to thesis writing, and to provide some more example material to give you a feeling for good typesetting.

2.1 Basic writing rules

The following rules need little further explanation; they are best understood by looking at the example in the [booklet by Knuth et al.](#), §2–§3.

Rule 2.1 Write texts, not chains of formulas.

More specifically, write full sentences that are logically interconnected by phrases like ‘Therefore’, ‘However’, ‘On the other hand’, etc. where appropriate.

Rule 2.2 Displayed formulas should be embedded in your text and punctuated with it.

In other words, your writing should not be divided into ‘text parts’ and ‘formula parts’; instead the formulas should be tied together by your prose such that there is a natural flow to your writing.

2.2 Being nice to the reader

Try to write your text in such a way that a reader enjoys reading it. That’s of course a lofty goal, but nevertheless one you should aspire to!

Rule 2.3 Be nice to the reader.

Give some intuition or easy example for definitions and theorems which might be hard to digest. Remind the reader of notations you introduced

many pages ago – chances are they have forgotten them. Illustrate your writing with diagrams and pictures where this helps the reader. Etc.

Rule 2.4 Organize your writing.

Think carefully about how you subdivide your thesis into chapters, sections, and possibly subsections. Give overviews at the beginning of your thesis and of each chapter, so the reader knows what to expect. In proofs, outline the main ideas before going into technical details. Give the reader the opportunity to ‘catch up with you’ by summing up your findings periodically.

Useful phrases: ‘So far we have shown that ...’, ‘It remains to show that ...’, ‘Recall that we want to prove inequality (7), as this will allow us to deduce that ...’, ‘Thus we can conclude that Next, we would like to find out whether ...’, etc.

Rule 2.5 Don’t say the same thing twice without telling the reader that you are saying it twice.

Repetition of key ideas is important and helpful. However, if you present the same idea, definition or observation twice (in the same or different words) without telling the reader, they will be looking for something new where there is nothing new.

Useful phrases: ‘Recall that [we have seen in Chapter 5 that] ...’, ‘As argued before / in the proof of Lemma 3, ...’, ‘As mentioned in the introduction, ...’, ‘In other words, ...’, etc.

Rule 2.6 Don’t make statements that you will justify later without telling the reader that you will justify them later.

This rule also applies when the justification is coming right in the next sentence! The reasoning should be clear: if you violate it, the reader will lose valuable time trying to figure out on his own what you were going to explain to him anyway.

Useful phrases: ‘Next we argue that ...’, ‘As we shall see, ...’, ‘We will see in the next section that ...’, etc.

2.3 A few important grammar rules

Rule 2.7 There is (almost) *never* a comma before ‘that’.

It’s really that simple. Examples:

We assume that ...

Wir nehmen an, dass ...

It follows that ...

Daraus folgt, dass ...

‘thrice’ is a word that is seldom used.

‘thrice’ ist ein Wort, das selten verwendet wird.

Exceptions to this rule are rare and usually pretty obvious. For example, you may end up with a comma before ‘that’ because ‘i.e.’ is spelled out as ‘that is’:

For $p(n) = \log n/n$ we have ... However, if we choose p a little bit higher, that is $p(n) = (1 + \varepsilon) \log n/n$ for some $\varepsilon > 0$, we obtain that...

Or you may get a comma before ‘that’ because there is some additional information inserted in the middle of your sentence:

Thus we found a number, namely n_0 , that satisfies equation (13).

If the additional information is left out, the sentence has no comma:

Thus we found a number that satisfies equation (13).

(For ‘that’ as a relative pronoun, see also Rules 2.9 and 2.10 below.)

Rule 2.8 There is usually no comma before ‘if’.

Example:

A graph is not 3-colorable if it contains a 4-clique.

Ein Graph ist nicht 3-färbbar, wenn er eine 4-Clique enthält.

However, if the ‘if’ clause comes first, it is usually separated from the main clause by a comma:

If a graph contains a 4-clique, it is not 3-colorable .

Wenn ein Graph eine 4-Clique enthält, ist er nicht 3-färbbar.

There are more exceptions to these rules than to Rule 2.7, which is why we are not discussing them here. Just keep in mind: don’t put a comma before ‘if’ without good reason.

Rule 2.9 Non-defining relative clauses have commas.

Rule 2.10 Defining relative clauses have no commas.

In English, it is very important to distinguish between two types of relative clauses: defining and non-defining ones. This is a distinction you absolutely need to understand to write scientific texts, because mistakes in this area actually distort the meaning of your text!

It’s probably easier to explain first what a *non-defining* relative clause is. A non-defining relative clause simply gives additional information *that could also be left out* (or given in a separate sentence). For example, the sentence

The WEIRDSORT algorithm, which was found by the famous mathematician John Doe, is theoretically best possible but difficult to implement in practice.

would be fully understandable if the relative clause were left out completely. It could also be rephrased as two separate sentences:

The WEIRDSORT algorithm is theoretically best possible but difficult to implement in practice. [By the way,] WEIRDSORT was found by the famous mathematician John Doe.

This is what a non-defining relative clause is. *Non-defining relative clauses are always written with commas.* As a corollary we obtain that you cannot use ‘that’ in non-defining relative clauses (see Rule 2.7!). It would be wrong to write

~~The WEIRDSORT algorithm, that was found by the famous mathematician John Doe, is theoretically best possible but difficult to implement in practice.~~

A special case that warrants its own example is when ‘which’ is referring to the entire preceding sentence:

Thus inequality (7) is true, which implies that the Riemann hypothesis holds.

As before, this is a non-defining relative sentence (it could be left out) and therefore needs a comma.

So let’s discuss *defining* relative clauses next. A defining relative clause tells the reader *which specific item the main clause is talking about*. Leaving it out either changes the meaning of the sentence or renders it incomprehensible altogether. Consider the following example:

The WEIRDSORT algorithm is difficult to implement in practice. In contrast, the algorithm that we suggest is very simple.

Here the relative clause ‘that we suggest’ cannot be left out – the remaining sentence would make no sense since the reader would not know which algorithm it is talking about. This is what a defining relative clause is. *Defining relative clauses are never written with commas.* Usually, you can use both ‘that’ and ‘which’ in defining relative clauses, although in many cases ‘that’ sounds better.

As a final example, consider the following sentence:

For the elements in $\mathcal{B}$ which satisfy property (A), we know that equation (37) holds.

This sentence does not make a statement about all elements in $\mathcal{B}$, only about those satisfying property (A). The relative clause is *defining*. (Thus we could also use ‘that’ in place of ‘which’.)

2.4. Things you (usually) don't say in English

Table 2.1: Things you (usually) don't say

It holds (that) ... (‘Equation (5) holds.’ is fine, though.)	We have ...	<i>Es gilt ...</i>
x fulfills property $\mathcal{P}$.	x satisfies property $\mathcal{P}$.	<i>x erfüllt Eigenschaft $\mathcal{P}$.</i>
in average	on average	<i>im Durchschnitt</i>
estimation	estimate	<i>Abschätzung</i>
composed number	composite number	<i>zusammengesetzte Zahl</i>
with the help of	using	<i>mit Hilfe von</i>
surely	clearly	<i>sicher, bestimmt</i>
monotonously increasing	monotonically incr.	<i>monoton steigend</i>
(Actually, in most cases ‘increasing’ is just fine.)		

In contrast, if we add a comma the sentence reads

For the elements in $\mathcal{B}$, which satisfy property (A), we know that equation (37) holds.

Now the relative clause is *non-defining* – it just mentions in passing that all elements in $\mathcal{B}$ satisfy property (A). The main clause states that equation (37) holds for *all* elements in $\mathcal{B}$. See the difference?

Rule 2.11 Don't write sentences like “Let N_1 respectively N_2 be the neighborhoods of the vertices v_1 respectively v_2 ”. While this may feel correct in German, the proper form in English would be “Let N_1 and N_2 be the neighborhoods of the vertices v_1 and v_2 , respectively”. By adding the word *respectively* in the end we mean that the mapping between N_1, N_2 and v_1, v_2 is a matching (in particular the one implied by the ordering). The proper translation of the English word “respectively” into German is “jeweils” (and not “respektive” or “beziehungsweise”), and the proper translation of the German word “respektive” into English is “and”.

2.4 Things you (usually) don't say in English – and what to say instead

Table 2.1 lists some common mistakes and alternatives. The entries should not be taken as gospel – they don't necessarily mean that a given word or formulation is wrong under all circumstances (obviously, this depends a lot on the context). However, in nine out of ten instances the suggested alternative is the better word to use.

Chapter 3

Typography

3.1 Punctuation

Rule 3.1 Use opening (‘) and closing (’) quotation marks correctly.

In L^AT_EX, the closing quotation mark is typed like a normal apostrophe, while the opening quotation mark is typed using the French *accent grave* on your keyboard (the *accent grave* is the one going down, as in *frère*).

Note that any punctuation that *semantically* follows quoted speech goes inside the quotes in American English, but outside in Britain. Also, Americans use double quotes first. Oppose

“Using ‘lasers,’ we punch a hole in ... the Ozone Layer,” Dr. Evil said.

to

‘Using “lasers”, we punch a hole in ... the Ozone Layer’, Dr. Evil said.

Rule 3.2 Use hyphens (-), en-dashes (–) and em-dashes (—) correctly.

A hyphen is only used in words like ‘well-known’, ‘3-colorable’ etc., or to separate words that continue in the next line (which is known as hyphenation). It is entered as a single ASCII hyphen character (-).

To denote ranges of numbers, chapters, etc., use an en-dash (entered as two ASCII hyphens --) with no spaces on either side. For example, using Equations (1)–(3), we see...

As the equivalent of the German *Gedankenstrich*, use an en-dash with spaces on both sides – in the title of Section 2.4, it would be wrong to use a hyphen instead of the dash. (Some English authors use the even longer emdash (—))

instead, which is typed as three subsequent hyphens in $\text{\LaTeX}$. This emdash is used without spaces around it—like so.)

3.2 Spacing

Rule 3.3 Do not add spacing manually.

You should never use the commands `\` (except within tabulars and arrays), `\_` (except to prevent a sentence-ending space after Dr. and such), `\vspace`, `\hspace`, etc. The choices programmed into $\text{\LaTeX}$ and this style should cover almost all cases. Doing it manually quickly leads to inconsistent spacing, which looks terrible. Note that this list of commands is by no means conclusive.

Rule 3.4 Judiciously insert spacing in maths where it helps.

This directly contradicts Rule 3.3, but in some cases $\text{\TeX}$ fails to correctly decide how much spacing is required. For example, consider

$$f(a,b) = f(a+b,a-b).$$

In such cases, inserting a thin math space `\,` greatly increases readability:

$$f(a,b) = f(a+b, a-b).$$

Along similar lines, there are variations of some symbols with different spacing. For example, Lagrange’s Theorem states that $|G| = [G : H]|H|$, but the proof uses a bijection $f: aH \rightarrow bH$. (Note how the first colon is symmetrically spaced, but the second is not.)

Another example is the following:

$$f(n) = n - 3, \quad n \geq 3.$$

Compare with the non-spaced version:

$$f(n) = n - 3, n \geq 3,$$

which is a bit difficult to parse. Some spacing command examples are given [here](#).

Rule 3.5 Learn when to use `\_` and `\@`.

Unless you use ‘french spacing’, the space at the end of a sentence is slightly larger than the normal interword space.

The rule used by $\text{\TeX}$ is that any space following a period, exclamation mark or question mark is sentence-ending, except for periods preceded by an upper-case letter. Inserting `\` before a space turns it into an interword space, and inserting `\@` before a period makes it sentence-ending. This means you should write

Prof.\ Dr.\ A. Steger is a member of CADMO\@.
 If you want to write a thesis with her, you
 should use this template.

which turns into

Prof. Dr. A. Steger is a member of CADMO. If you want to write
 a thesis with her, you should use this template.

The effect becomes more dramatic in lines that are stretched slightly during justification:

Prof. Dr. A. Steger is a member of CADMO. If you

Rule 3.6 Place a non-breaking space (~) right before references.

This is actually a slight simplification of the real rule, which should invoke common sense. Place non-breaking spaces where a line break would look ‘funny’ because it occurs right in the middle of a construction, especially between a reference type (Chapter) and its number.

3.3 Choice of ‘fonts’

Professional typography distinguishes many font attributes, such as family, size, shape, and weight. The choice for sectional divisions and layout elements has been made, but you will still occasionally want to switch to something else to get the reader’s attention. The most important rule is very simple.

Rule 3.7 When emphasising a short bit of text, use `\emph`.

In particular, *never* use bold text (`\textbf`). Italics (or Roman type if used within italics) avoids distracting the eye with the huge blobs of ink in the middle of the text that bold text so quickly introduces.

Occasionally you will need more notation, for example, a consistent typeface used to identify algorithms.

Rule 3.8 Vary one attribute at a time.

For example, for WEIRDSORT we only changed the shape to small caps. Changing two attributes, say, to bold small caps would be excessive ($\text{\LaTeX}$ does not even have this particular variation). The same holds for mathematical notation: the reader can easily distinguish g_n , $G(x)$, $\mathcal{G}$ and G .

Rule 3.9 Never underline or uppercase.

No exceptions to this one, unless you are writing your thesis on a typewriter. Manually. Uphill both ways. In a blizzard.

3.4 Displayed equations

Rule 3.10 Insert paragraph breaks *after* displays only where they belong. Never insert paragraph breaks *before* displays.

L^AT_EX translates sequences of more than one linebreak (i.e., what looks like an empty line in the source code) into a paragraph break in almost all contexts. This also happens before and after displays, where extra spacing is inserted to give a visual indication of the structure. Adding a blank line in these places may look nice in the sources, but compare the resulting display

$$a = b$$

to the following:

$$a = b$$

The first display is surrounded by blank lines, but the second is not. It is bad style to start a paragraph with a display (you should always tell the reader what the display means first), so the rule follows.

Rule 3.11 Never use eqnarray.

It is at the root of most ill-spaced multiline displays. The *amsmath* package provides better alternatives, such as the `align` family

$$\begin{aligned} f(x) &= \sin x, \\ g(x) &= \cos x, \end{aligned}$$

and `multline` which copes with excessively long equations:

$$\begin{aligned} \mathbb{P}[X_{t_0} \in (z_0, z_0 + dz_0], \dots, X_{t_n} \in (z_n, z_n + dz_n)] \\ = \nu(dz_0)K_{t_1}(z_0, dz_1)K_{t_2-t_1}(z_1, dz_2) \cdots K_{t_n-t_{n-1}}(z_{n-1}, dz_n). \end{aligned}$$

Notice how for `align` the different equations are aligned where the `&` occurs. For `multline`, one adds a line break (`\\`) and the rest is handled by the package.

A thing to keep in mind when using multi-line displays is to try and keep them short. A rule of thumb is to not go above 3 lines in a single display. It is better to split them up with some explanatory text. For example, compare:

$$\begin{aligned} \text{Var}[X] &= \mathbb{E}[(X - \mathbb{E}[X])^2] \\ &= \mathbb{E}[(X - \mathbb{E}[X])(X - \mathbb{E}[X])] \\ &= \mathbb{E}[X^2 - 2X\mathbb{E}[X] + \mathbb{E}[X]^2] \\ &= \mathbb{E}[X^2] - \mathbb{E}[2X\mathbb{E}[X]] + \mathbb{E}[\mathbb{E}[X]^2] \\ &= \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[X] + \mathbb{E}[X]^2 \\ &= \mathbb{E}[X^2] - \mathbb{E}[X]^2. \end{aligned}$$

with the following version which splits the display and uses text to explain the steps. First, we manipulate the expression inside the variance like so:

$$\begin{aligned}\text{Var}[X] &= \mathbb{E}[(X - \mathbb{E}[X])^2] \\ &= \mathbb{E}[(X - \mathbb{E}[X])(X - \mathbb{E}[X])] \\ &= \mathbb{E}[X^2 - 2X\mathbb{E}[X] + \mathbb{E}[X]^2].\end{aligned}$$

At this point, we can use linearity of expectation twice and some simple algebra to finally derive:

$$\begin{aligned}\text{Var}[X] &= \mathbb{E}[X^2] - \mathbb{E}[2X\mathbb{E}[X]] + \mathbb{E}[\mathbb{E}[X]^2] \\ &= \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[X] + \mathbb{E}[X]^2 \\ &= \mathbb{E}[X^2] - \mathbb{E}[X]^2.\end{aligned}$$

3.5 Floats

By default this style provides floating environments for tables and figures. The general structure should be as follows:

```
\begin{figure}
  \centering
  % content goes here
  \caption{A short caption}
  \label{some-short-label}
\end{figure}
```

Note that the label must follow the caption, otherwise the label will refer to the surrounding section instead. Also note that figures should be captioned at the bottom, and tables at the top.

The whole point of floats is that they, well, *float* to a place where they fit without interrupting the text body. This is a frequent source of confusion and changes; please leave it as is.

Rule 3.12 Do not restrict float movement to only ‘here’ (h).

If you are still tempted, you should avoid the float altogether and just show the figure or table inline, similar to a displayed equation.

More Tips on Math/L^AT_EX

4.1 The basics

Rule 4.1 (Display math) You might have heard at some point that it is not a great idea to use `$$...$$`. Indeed, the preferred way is to use `\[...\]` or equivalently `\begin{displaymath}...\end{displaymath}`. The problem with the former is some issues with spacing and also that the command `\fleqn` (which flushes math to the left) does not work.

Rule 4.2 (Punctuation) Speaking about inline math and display math, keep in mind that one should have punctuation outside of inline math, but inside display math. That is, we say: The random variable X , which has expectation 5, is positive. But we would say: the expectation of the random variable X , which satisfies

$$X > 0,$$

is 5.

Rule 4.3 (Roman font) For functions like $\log n$, $\cos n$, don't just write their name in math, i.e. $\log n$, $\cos n$. Instead, use the macros when available (as here), or if they are not you can use `\mathrm{function}`, which prints function. This uses the roman font in math. Note that this is also what we have done for defining the `\Var` macro.

Rule 4.4 (Dots) Do not use `$(x_1, \dots, x_n)$`. Instead, you should use `$(x_1, \ldots, x_n)$`, i.e. $(x_1, \dots, x_n)$ when you want lowered dots and `$A \times \cdots \times Z$` ($A \times \cdots \times Z$) when you want centered dots.

4.2 Some common math for reference/uniformity

Rule 4.5 (Set building) A very common thing to do is define sets via some sort of condition, like so:

$$X = \{ x \in \mathbb{R} : f(x) > 0 \}.$$

You can use our macro `\Set` for this, which takes two arguments, the left and right-hand side of the `:` symbol. You can also replace this with `\mid ()`, but make sure to be consistent.

Rule 4.6 (Cases) Sometimes we want to define a piecewise function. This can be done with the `\cases` environment.

$$f(n) = \begin{cases} n, & \text{if } n < T/2 \\ T - n, & \text{otherwise} \end{cases}$$

Rule 4.7 (Matrices) Matrices are also super common, here are some examples:

$$\begin{pmatrix} 1 & 2 & 3 \\ a & b & c \end{pmatrix}, \quad \begin{bmatrix} 1 & 2 & 3 \\ e & f & g \end{bmatrix}, \quad \begin{Bmatrix} 1 & 2 & 3 \\ h & i & j \end{Bmatrix}$$

One thing that you might not know is that you can have small matrices for inline math, like so: $\begin{pmatrix} k & l \\ m & n \end{pmatrix}$.

Rule 4.8 (Binomials) Finally, it is highly likely that you use binomial coefficients somewhere in your thesis, like so:

$$\binom{n}{k}.$$

4.3 Figures and Algorithms

Almost always it is a good idea to use figures to illustrate your arguments and help the reader understand them more deeply. For this, you can use TikZ, documented [here](#). Below we give an example figure (Figure 4.1) with some elements that might be useful. For other examples, look at the documentation (it is actually good and not just incomprehensible walls of text, i.e. it also has examples you can modify to your needs easily).

Also, describing algorithms is something you most likely will have to do. We suggest using the `algorithmicx`, `algpseudocode` packages for this. Below (Algorithm 1) is an example.

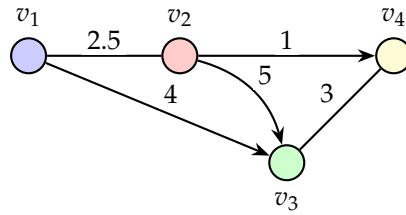


Figure 4.1: Example of a directed weighted graph using TikZ. Some edges are drawn undirected.

Algorithm 1 Sample Algorithm

```

procedure EXAMPLE( $A$ )
   $x \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $n$  do
    if  $A[i] > x$  then
       $x \leftarrow A[i]$ 
    end if
  end for
  return  $x$ 
end procedure

```

4.4 Other tips and tricks

Now we will see some less common but nevertheless useful things one can do with $\text{\LaTeX}$. Perhaps the most important of these is the use of *restatable* theorem environments.

Rule 4.9 (Restatables) An academic paper typically has a “Results” section where the authors present the main theorems. Their proof is usually much later in the paper, and it helps to have the statement of the theorem before the proof as well. However, if we simply copy paste the theorem, it would get a new number in the second occurrence, as shown below.

Theorem 4.10 *This is a theorem with inconsistent numbering.*

Theorem 4.11 *This is a theorem with inconsistent numbering.*

The solution is in the `thm-restate` package. Below we show how to use it, simply look at the code.

Theorem 4.12 *I can restate this.*

Now that we wrote the theorem for the first time, we can restate it and write its proof.

Theorem 4.12 *I can restate this.*

Proof One can see. □

Rule 4.13 (Fractions and vertical spacing) It is very common to overuse the `\frac{}{}` command. However, this creates some issues, especially when it is used in exponents of inline math. Notice the weird vertical spacing when we use $e^{\frac{n^2}{8}}$. This happens because L^AT_EX surrounds the expression with a “box” which pushes the line downwards so there is no overlap. A (partial) solution is to make this box shorter, by using for example $e^{n^2/8}$ instead. Or even $\exp(n^2/8)$ to make the effect disappear completely. In general, try to limit the use of “tall” math inline, to maintain vertical spacing consistent. This also applies to the overuse of `\left(\dots\right)`, which might size the parentheses a bit too tall. This can happen implicitly if you overuse the starred version of the macros for Landau notation, et cetera. When using inline math, always choose the bracket sizes manually. While we are at it, here is another tip on `\frac{}{}`. There are also the commands `\tfrac{}{}` and `\dfrac{}{}`, which stand for text style (inline math) and display style (display math). Using `\dfrac{}{}` in inline math makes the fraction larger: $\frac{22}{7}$. The functionality of `\tfrac{}{}` is the reverse inside display math.

Rule 4.14 (Referring to equations) You can use the `equation` environment to write down an important equation, numbering it so that you can refer to it later.

$$E = mc^2 \tag{4.1}$$

Who does not know eq. (4.1)? Excuse me, I meant Equation (4.1)! Instead of using a number, one can also *tag* an equation with a name.

$$F = ma \tag{force}$$

Equation (force) is important too!

Rule 4.15 (Detexify) Sometimes you just can’t remember or find the latex code for a symbol (although both the Overleaf and VSCode extension widgets work well for this). Thankfully, there is a tool online where you can draw a math expression with your mouse and it gives you a list of candidate L^AT_EX commands. [Here](#) it is.

Rule 4.16 (Conditional L^AT_EX) L^AT_EX has a neat figure that allows you to set flags and include content conditionally on those flags. One of the most important use cases of this is when sending an article for anonymous review. One can set an anon flag based on which their name may be hidden from the final output. We give an example here:

This manuscript was written by A. Uthor. Now I cannot tell you that ANON wrote the manuscript.

How a Typical Academic Paper is Structured

Before delving into a concrete example, it is useful to understand the common structure of an academic paper—especially in the fields of computer science and mathematics. While specific requirements can vary across disciplines, journals, or conferences, most technical papers and theses follow a broadly similar organizational pattern. This chapter serves as an overview of that structure, highlighting the purpose and typical contents of each major section.

5.1 Preamble

- **Title:** A concise statement reflecting the main topic or contribution. It should be both informative and engaging, often hinting at the central findings or approach.
- **Acknowledgments:** A short section/chapter to recognize the individuals, organizations, or funding agencies that supported the research. Some publications (conference or journal styles) place this section at the end, while others place it in a footnote on the first page.
- **Abstract:** A brief (usually 150–300 words) summary of the paper. The abstract succinctly states the *problem*, the *approach* used to tackle the problem, and the *main results/contributions*. Readers often decide whether to read the entire paper based on the clarity and relevance of the abstract.

5.2 Introduction

The introduction typically provides:

- **Context and Motivation:** Explains why the topic is important, giving background or real-world relevance. The existence of many previous works studying related questions is also sometimes used as further motivation.
- **Problem Statement:** Defines the core question or problem addressed by the paper.
- **Prior Work or Background:** Summarizes existing literature and situates the problem within a broader research context. (Some papers have a separate “Related Work” section for more extensive discussion of prior research, if necessary.)
- **Contributions and Overview:** Concludes by stating what new insight, technique, or theoretical result the paper offers (listing the theorems, possibly restatable), and outlines the paper’s structure to guide readers through the subsequent sections.

5.3 Notation and Preliminaries

- **Notation and Definitions:** Introduces the notation, terminology, or foundational concepts essential for understanding the paper’s main results.
- **Theoretical Tools:** Reviews any theorems, lemmas, or existing results (from probability, linear algebra, combinatorics, etc.) that the paper will rely on.
- **Contextual Details:** In some fields, this section might include domain-specific information (e.g., details on data models, assumptions about hardware, or historical context). Here you might also more formally define the problem/question studied, in case you opted for a more intuitive explanation in the introduction (which is a good approach).

5.4 Main Content (Approach/Analysis)

This is usually the largest and most detailed section. Depending on the nature of the work, it might be subdivided in various ways:

5.4.1 Algorithm or Method Description

- **High-Level Idea:** An intuitive, conceptual overview of the proposed algorithm or approach.

- **Pseudocode or Formal Specification:** A step-by-step procedure or equations describing how the algorithm operates or how the proof of the result is structured.

5.4.2 Theoretical Analysis

- **Correctness/Runtime/Theorem Proofs:** Here the formal proofs are given in all detail. Typically there is still a storyline to be followed in text to not lose the reader in the math, but each proof environment is now fully formal and self-standing. You can think of the explanations along the way like comments in code and the theorems/proofs as the actual code, which has to follow strict rules.

5.4.3 Experimental Results (If Applicable)

For more empirically focused papers:

- **Dataset Description:** Overview of the data used for experiments, how it was collected, and why it is suitable.
- **Metrics:** Explanation of how performance or accuracy is measured.
- **Comparison with Baselines:** Charts, tables, or graphs illustrating how the proposed method stacks up against existing methods.

5.5 Conclusion and Future Work

The conclusion typically:

- **Summarizes Main Findings:** Restates the key contributions and what has been learned.
- **Discusses Limitations:** Briefly considers conditions or scenarios where the proposed method might not perform as well.
- **Suggests Future Directions:** Outlines open questions or new problems raised by the work, offering pathways for continued research.

5.6 References

- **Citations:** Every work mentioned in the paper (including background literature, data sources, and tools) should be cited in a consistently formatted bibliography.
- **Formatting Style:** Whether it's APA, IEEE, ACM, Chicago, or another style, references should follow a recognized standard and be clear to

the reader. For the thesis (and most CS conferences), you can use the citing style found in this template.

5.7 Appendices (If Needed)

- **Supplementary Material:** Large/overly technical proofs, extended data tables, or technical details that disrupt the narrative flow can be placed in appendices to keep the main text concise.
- **Extended Examples:** Additional or more detailed examples that illustrate nuances of the proposed approach.

In This Template

In the chapters that follow, we demonstrate how these broad guidelines translate into a concrete academic document. You will see:

- An **Abstract** that briefly states the problem, the approach, and the main findings.
- An **Introduction** providing motivation and an overview of prior work.
- A chapter on **Notation and Preliminaries** that sets up key definitions and outlines the main probabilistic tool—Chebyshev’s inequality—used in the analysis.
- A detailed **Algorithm and Analysis** section illustrating the design of a randomized median-finding procedure and its theoretical guarantees.
- A concise **Conclusion** summarizing contributions and suggesting future research directions.

This organization—while slightly customized for our particular focus on randomized selection algorithms—follows the same general principles outlined above. Now that we have an overview of the typical structure, we can move on to our specific example of how these elements come together in a coherent academic work. Note that by default the abstract and title of the “paper” are not shown. To see them (and also start filling in your thesis content), set the `\ifstudentwriting` flag to true in `thesis.tex`.

Chapter 6

Introduction

The median of a dataset is a fundamental measure of central tendency that arises in numerous computational and statistical contexts. In an array of n elements, indexed from 1 to n , the median can be intuitively understood as the “middle” element in the sorted order if n is odd, or the average of the two middle elements if n is even. While this definition naturally suggests sorting the entire array, doing so in $\mathcal{O}(n \log(n))$ time is often unnecessary when the goal is to identify only the median.

Motivation and Previous Work

Several well-known algorithms exist for selecting the median or, more generally, the k -th smallest element of an array. A straightforward method is to sort the array and then pick the median in constant time. However, this approach has a runtime of $\mathcal{O}(n \log(n))$, which can be excessive for large n . More advanced methods, such as the *median-of-medians* algorithm introduced by Blum, Floyd, Pratt, Rivest, and Tarjan [1], achieve a worst-case runtime of $\mathcal{O}(n)$ using a deterministic divide-and-conquer strategy. While optimal in the worst case, the median-of-medians algorithm involves significant overhead, making it less practical in many scenarios. Alternatively, randomized algorithms like Quickselect often achieve linear-time performance on average but can degrade to $\mathcal{O}(n^2)$ in the worst case without additional precautions.

Our Contribution

In this thesis, we present a *randomized* algorithm that computes the median in $\mathcal{O}(n)$ time *with high probability*, thereby combining practical efficiency with strong theoretical guarantees. The key idea is to leverage a carefully chosen random sample of the input array:

-
1. We extract a subset of elements (the “sample”) whose size grows sub-linearly with n .
 2. Using this sample, we identify pivot elements that, with high probability, closely approximate the true median’s position.
 3. We then reduce the problem to sorting only a small subset of the original array, thereby achieving near-linear time complexity.

Central to our analysis is the use of **Chebyshev’s inequality**. While often introduced in purely statistical contexts, Chebyshev’s inequality provides a useful probabilistic bound for ensuring that the sampled elements approximate the median accurately. In essence, it tells us that $\mathbb{P}[|X - \mathbb{E}[X]| \geq k]$ can be kept small if $\text{Var}[X]$ is finite, thereby giving us high confidence that our pivot-based “zoom-in” on the median is correct.

More concretely, we prove the following theorem with our algorithm.

Theorem 6.1 *There exists a randomized algorithm to compute the median that runs in time $\mathcal{O}(n)$ and succeeds with probability $1 - \mathcal{O}(n^{-1/4})$.*

Thesis Outline

This thesis is structured as follows:

- **Chapter 7 (Notation and Preliminaries):** We define our notational conventions and offer a brief refresher on Chebyshev’s inequality, setting the stage for the randomized analysis to come.
- **Chapter 8 (Algorithm and Analysis):** We detail the randomized median-finding algorithm, present its pseudocode, and provide a rigorous proof of correctness. We also analyze the runtime and show why it remains $\mathcal{O}(n)$ with high probability.
- **Chapter 9 (Conclusion):** We summarize our findings, discuss the algorithm’s theoretical and practical implications, and highlight possible directions for future research.

Notation and Preliminaries

Definitions and Notation

Arrays and Median

Throughout this thesis, we denote the input array of size n by

$$a = (a[1], a[2], \dots, a[n]).$$

We may also use $a[1 \dots n]$ to denote an array a of length n . Our goal is to find the *median* of a . Formally, we define:

- If n is odd, the median is the element in the $\lceil n/2 \rceil$ -th position of the array when sorted in non-decreasing order.
- If n is even, the median is the element in the $\lceil n/2 \rceil$ -th position of the sorted array.

Random Variables and Expectation

In the analysis of our randomized median-finding algorithm, we shall frequently refer to *random variables*. A random variable X is a function from the sample space Ω (representing all possible outcomes of a probabilistic experiment) to the real numbers $\mathbb{R}$.

Expectation. The *expected value*, or *mean*, of X is defined by

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega) \mathbb{P}[\omega],$$

in the discrete case. If X has a probability density function f_X (in the continuous case), then

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x f_X(x) dx.$$

Intuitively, $\mathbb{E}[X]$ indicates the “average” or “central” value of the random variable over many independent trials.

Variance and Standard Deviation

To measure how much a random variable deviates from its mean, we use its *variance*, denoted $\text{Var}[X]$. This is defined as:

$$\text{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

The *standard deviation* of X , denoted σ_X , is the square root of the variance:

$$\sigma_X = \sqrt{\text{Var}[X]}.$$

A smaller variance (or standard deviation) means the random variable’s values tend to lie closer to its mean, whereas a larger variance indicates a broader spread of values.

Asymptotic Notation

In describing the time complexity of algorithms, we use $\mathcal{O}(\dots)$ notation in a standard way:

$$f(n) = \mathcal{O}(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty.$$

We may also use $\Theta(\dots)$, $\Omega(\dots)$, and other asymptotic notations where necessary.

Chebyshev’s Inequality

Chebyshev’s inequality is a cornerstone result in probability theory, providing a quantitative bound on how a random variable deviates from its expected value. Formally, let X be any random variable with finite mean $\mathbb{E}[X]$ and finite variance $\text{Var}[X]$. Then, for any $k > 0$,

$$\mathbb{P}[|X - \mathbb{E}[X]| \geq k] \leq \frac{\text{Var}[X]}{k^2}.$$

Intuition and Relation to Markov’s Inequality

To build intuition, recall the simpler *Markov’s inequality*, which states that for any nonnegative random variable Y and $t > 0$,

$$\mathbb{P}[Y \geq t] \leq \frac{\mathbb{E}[Y]}{t}.$$

Chebyshev's inequality is often viewed as an application of Markov's inequality to the random variable $(X - \mathbb{E}[X])^2$. Indeed,

$$\mathbb{P}[(X - \mathbb{E}[X])^2 \geq k^2] \leq \frac{\mathbb{E}[(X - \mathbb{E}[X])^2]}{k^2} = \frac{\text{Var}[X]}{k^2},$$

and from $\{|X - \mathbb{E}[X]| \geq k\} \equiv \{(X - \mathbb{E}[X])^2 \geq k^2\}$, we obtain Chebyshev's inequality.

The key takeaway is that Chebyshev's inequality captures a broad class of distributions. It requires no assumption of normality or other specific distributional shapes, only finite variance. In exchange for this generality, Chebyshev's bound can be somewhat loose compared to more specialized inequalities (e.g., Hoeffding's, Chernoff's, or Bernstein's). Nonetheless, it remains a powerful and classic tool for bounding "tail events," i.e. events in which X significantly deviates from its mean.

Why We Use Chebyshev's Inequality Here

In our randomized median-finding algorithm, we draw random samples from the input array and use them to approximate the true median. The "rank" of our chosen pivots in the sorted order of the entire array are themselves random variables. We want to show that, with high probability, these rank contain the $\lceil n/2 \rceil$ -th position.

By applying Chebyshev's inequality, we obtain a bound on $\mathbb{P}[|X - \mathbb{E}[X]| \geq k]$ for a well-chosen k , where X is the rank of our pivot. In other words, Chebyshev's inequality formally establishes the reliability of our sampling strategy by demonstrating that, except with small probability, our pivots partition the array around the actual median. This allows us to restrict further computation to a subset of the array whose size is guaranteed to be sufficiently small, thereby ensuring linear running time.

Extensions and Alternatives

While Chebyshev's inequality is sufficient for our purposes, more refined analyses might employ other concentration bounds such as Hoeffding's or Chernoff's inequalities. These stronger bounds can yield tighter probabilities when the underlying sample distribution has additional regularity properties (e.g., bounded random variables or independent Bernoulli trials). However, Chebyshev's inequality has the advantage of simplicity and broad applicability, making it an ideal choice for illustrating the core ideas in a clear manner.

Algorithm and Analysis

In this chapter, we present our main randomized median-finding algorithm in full detail and analyze its performance. We first give a high-level description and provide the pseudocode in Algorithm 2. We then prove two central theorems: one that establishes the algorithm's probability of success (i.e. returning the correct median) and another that bounds its running time by $\mathcal{O}(n)$. Throughout this chapter, we use the notation introduced in Chapter 7.

Algorithm Description

The guiding idea behind the algorithm is to select a “small” random sample from the input array, sort this sample, and use carefully chosen elements from this sample as *pivots* to bracket the true median. Specifically:

- We draw $m = \lceil n^{3/4} \rceil$ elements uniformly at random from the input array $a[1 \dots n]$ and sort these sampled elements into an array $b[1 \dots m]$.
- We then pick two elements from b , denoted p_ℓ and p_r , near the middle of b but offset by $\pm\sqrt{n}$. Intuitively, these two pivots should “sandwich” the true median of the entire array with high probability.
- We gather all elements of the original array a that lie between these two pivots (inclusive) into a temporary array T . We verify that the median must lie in this sub-array, and that T is not excessively large.
- If those checks are satisfied, we sort T and extract the median; otherwise, the algorithm reports a *failure*.

The rest of this chapter gives a proof of two properties of this algorithm:

1. With high probability, Algorithm 2 does not fail and returns the correct median.
2. Conditioned on success, the overall running time is $\mathcal{O}(n)$.

Algorithm 2 RandMedian: Randomized Median Algorithm

Require: Unsorted array $a[1 \dots n]$ of size n .

Ensure: Median of a or *failure*.

- 1: **Initialize** an empty array b .
- 2: **For each** element $x \in a[1 \dots n]$:
 Include x in b **with probability** $n^{-1/4}$, independently of other elements.
- 3: $m \leftarrow |b|$ $\triangleright$ (Random) size of the sample; note that $\mathbb{E}[m] = n^{3/4}$.
- 4: **if** $m > n^{3/4} + \sqrt{n}$ **then**
- 5: **return** *failure*
- 6: **end if**
- 7: **Sort** b in $\mathcal{O}(m \log m)$ time
- 8: **Compute pivots:**

$$p_\ell \leftarrow b \left[\left\lfloor \frac{m}{2} - \sqrt{n} \right\rfloor \right], \quad p_r \leftarrow b \left[\left\lceil \frac{m}{2} + \sqrt{n} \right\rceil \right]$$

- 9: Define the subset $T \leftarrow \{x \in a : p_\ell \leq x \leq p_r\}$

- 10: Compute:

$$s \leftarrow \#\{x \in a : x < p_\ell\},$$
$$t \leftarrow \#\{x \in a : p_\ell \leq x \leq p_r\}.$$

- 11: **if** $s < \lceil n/2 \rceil$ **and** $s + t \geq \lceil n/2 \rceil$ **and** $t < 4n^{3/4}$ **then**
 - 12: **Sort** T in $\mathcal{O}(|T| \log |T|)$ time
 - 13: Median of $a \leftarrow T[\lceil n/2 \rceil - s]$
 - 14: **return** Median of a
 - 15: **else**
 - 16: **return** *failure*
 - 17: **end if**
-

Probability of Success

We begin by showing that, except with small probability, the two pivots p_ℓ and p_r are chosen so that the sub-array T contains the true median of a and is not excessively large. Formally, we have:

Theorem 8.1 (Correctness with High Probability) *The probability that Algorithm 2 returns the correct median is at least $1 - \mathcal{O}(n^{-1/4})$. In other words, the algorithm succeeds with high probability.*

Proof For simplicity, we assume throughout this proof that n is even and that $n^{3/4}$ is an integer; this avoids cumbersome floor and ceiling functions. The general case follows by analogous, but slightly more tedious, arguments.

Recall that $a[1 \dots n]$ is our input array of n distinct elements. Let $b[1 \dots m]$ be the sample of size m drawn by sampling each element with probability

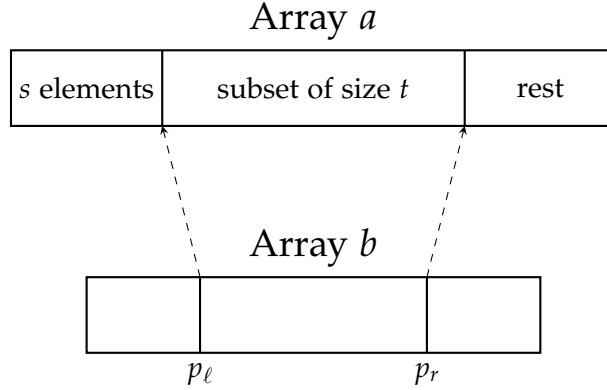


Figure 8.1: Illustration of the arrays and the pivots, showing array a , sampled array b , and the pivots p_ℓ, p_r . Note that this visualization shows the sorted version of the arrays.

$n^{-1/4}$ from a . The setup is also visualized in Figure 8.1. We then sort b , pick the pivots¹

$$p_\ell = b\left[\frac{m}{2} - \sqrt{n}\right], \quad p_r = b\left[\frac{m}{2} + \sqrt{n}\right],$$

and collect into T all elements $x \in a$ with $p_\ell \leq x \leq p_r$. Let

$$s = \#\{x \in a : x < p_\ell\} \quad \text{and} \quad t = \#\{x \in a : p_\ell \leq x \leq p_r\}.$$

Key sets and random variables. For any integer $k \in \{0, \dots, n\}$, define

$$S_k = \{\text{the } k \text{ smallest elements of } a\}.$$

We then define the random variable

$$X_k = \#\{x \in b : x \in S_k\}.$$

Since each element of a is independently included in b with probability $n^{-1/4}$, the number of elements of S_k that appear in b follows

$$X_k \sim \text{Bin}(k, n^{-1/4}).$$

Hence,

$$\mathbb{E}[X_k] = k n^{-1/4}, \quad \text{Var}[X_k] = k n^{-1/4}(1 - n^{-1/4}) \leq k n^{-1/4}.$$

Application of Chebyshev's Inequality. Since $X_k \sim \text{Bin}(k, n^{-1/4})$ with $\text{Var}[X_k] \leq k n^{-1/4}$, Chebyshev's inequality gives

$$\mathbb{P}[|X_k - k n^{-1/4}| \geq \sqrt{n}] \leq \frac{\text{Var}[X_k]}{n} \leq \frac{k n^{-1/4}}{n} = \frac{k}{n^{5/4}}. \quad (8.1)$$

¹Technically, if m is too small, the following lines are undefined (out of bounds). But this is avoided whp, as for the other events later in the proof.

Defining the “bad” events. We show that the only way the algorithm can fail (i.e. not return the correct median) is if one of the following four events occurs. Denote

$$\begin{aligned} A &:= \{s \geq n/2\}, \\ B &:= \{s + t < n/2\}, \\ C &:= \{s < \frac{n}{2} - 2n^{3/4}\}, \\ D &:= \{s + t \geq \frac{n}{2} + 2n^{3/4}\}, \\ E &:= \{m > n^{3/4} + \sqrt{n}\}. \end{aligned}$$

Notice that E not occurring means that the first “return failure” line is not executed. If none of these events happens, then we must have

$$s < \frac{n}{2}, \quad s + t \geq \frac{n}{2}, \quad s \geq \frac{n}{2} - 2n^{3/4}, \quad \text{and} \quad s + t < \frac{n}{2} + 2n^{3/4}.$$

One can check that these conditions imply exactly

$$(s < \frac{n}{2}) \quad \text{and} \quad (s + t \geq \frac{n}{2}) \quad \text{and} \quad t < 4n^{3/4},$$

so the algorithm’s final check (line 11 of the pseudocode) succeeds. In that case, our pivot-based bracket $[p_\ell, p_r]$ indeed contains the median, and the algorithm returns the correct value.

Probability of each bad event. We next prove that $\mathbb{P}[A], \mathbb{P}[B], \mathbb{P}[C], \mathbb{P}[D], \mathbb{P}[E]$ are each at most $n^{-1/4}$. Observe:

- Event A means $s \geq n/2$. Hence among the $n/2$ smallest elements of a (i.e. the set $S_{n/2}$), at most $\frac{1}{2}n^{3/4} - \sqrt{n}$ of them are drawn into b . In other words,

$$X_{n/2} \leq \frac{1}{2}n^{3/4} - \sqrt{n}.$$

But from $\mathbb{E}[X_{n/2}] = (n/2)n^{-1/4} = \frac{1}{2}n^{3/4}$ and (8.1), we get $\mathbb{P}[X_{n/2} \leq \frac{1}{2}n^{3/4} - \sqrt{n}] \leq n^{-1/4}$. Thus $\mathbb{P}[A] \leq n^{-1/4}$.

- Event B means $s + t < n/2$. Equivalently, the set $S_{n/2}$ has more than $\frac{1}{2}n^{3/4} + \sqrt{n}$ elements contained in b . Hence

$$X_{n/2} \geq \frac{1}{2}n^{3/4} + \sqrt{n}.$$

Again, from $\mathbb{E}[X_{n/2}] = \frac{1}{2}n^{3/4}$ and (8.1), we get $\mathbb{P}[X_{n/2} \geq \frac{1}{2}n^{3/4} + \sqrt{n}] \leq n^{-1/4}$. So $\mathbb{P}[B] \leq n^{-1/4}$.

- Event C means $s < n/2 - 2n^{3/4}$. In this case, the set $S_{n/2-2n^{3/4}}$ must have at least $\frac{1}{2}n^{3/4} - \sqrt{n}$ of its elements in b . Formally,

$$X_{n/2-2n^{3/4}} \geq \frac{1}{2}n^{3/4} - \sqrt{n}.$$

Using $\mathbb{E}[X_{n/2-2n^{3/4}}] = (n/2 - 2n^{3/4})n^{-1/4} = \frac{1}{2}n^{3/4} - 2\sqrt{n}$ and (8.1) again shows $\mathbb{P}[C] \leq n^{-1/4}$.

-
- Event D means $s + t \geq n/2 + 2n^{3/4}$. Hence the set $S_{n/2+2n^{3/4}}$ contains at most $\frac{1}{2}n^{3/4} + \sqrt{n}$ elements from b . That is,

$$X_{n/2+2n^{3/4}} \leq \frac{1}{2}n^{3/4} + \sqrt{n}.$$

A final application of (8.1) yields $\mathbb{P}[D] \leq n^{-1/4}$.

- A similar calculation involving X_n shows that $\mathbb{P}[E] \leq n^{-1/4}$.

Conclusion. By the union bound,

$$\mathbb{P}[A \cup B \cup C \cup D \cup E] \leq \mathbb{P}[A] + \mathbb{P}[B] + \mathbb{P}[C] + \mathbb{P}[D] + \mathbb{P}[E] \leq 5 \times n^{-1/4}$$

In other words, with probability at least $1 - 5n^{-1/4}$, none of these events occurs. When all five do not occur, the algorithm's checks

$$s < \lceil n/2 \rceil, \quad s + t \geq \lceil n/2 \rceil, \quad t < 4n^{3/4}$$

are satisfied, so the algorithm terminates successfully with the correct median. Hence the overall success probability is at least $1 - 5n^{-1/4}$, completing the proof. $\square$

Runtime Analysis

Next, we establish that Algorithm 2 achieves linear time complexity *with high probability*. Note that if the algorithm returns *failure*, it may be retried or combined with a fallback procedure. Here, we focus on the typical execution that succeeds.

Theorem 8.2 (Runtime) *Conditioned on the event that Algorithm 2 does not fail (an event which, by Theorem 8.1, holds with high probability), the total running time is $\mathcal{O}(n)$.*

Proof We break the cost into four parts:

1. **Sampling and Sorting b .** We pick $m \leq n^{3/4} + \sqrt{n} \leq 2n^{3/4}$ elements randomly from a (conditioned on success). This is done in $\mathcal{O}(n)$ time. Sorting b of size m then takes

$$\mathcal{O}(m \log(m)) = \mathcal{O}\left(n^{3/4} \log\left(n^{3/4}\right)\right).$$

Since $n^{3/4} \log(n^{3/4})$ is asymptotically smaller than n for large n , this component is $o(n)$.

2. **Computing Ranks and Sub-array T .** We determine how many elements of a lie below p_ℓ , how many lie above p_r , etc. This amounts to a single pass through a , comparing each element of a to p_ℓ and p_r . Hence, it costs $\mathcal{O}(n)$ time.

-
3. **Sorting T .** By Algorithm 2, we only proceed to sort T if $t < 4n^{3/4}$. Thus,

$$t \leq 4n^{3/4}.$$

Sorting T via a standard comparison-based algorithm costs

$$\mathcal{O}(|T| \log(|T|)) \leq \mathcal{O}\left(n^{3/4} \log\left(n^{3/4}\right)\right),$$

which again is $o(n)$.

4. **Extracting the Median.** After sorting T , we simply pick the element of T whose rank corresponds to $\lceil n/2 \rceil - s$. This is clearly an $\mathcal{O}(1)$ operation.

Summing these components, the *dominant term* is $\mathcal{O}(n)$ (from the pass to identify s and t). The sublinear sorting costs on b and on T both constitute $\mathcal{O}(n^{3/4} \log(n))$, which is strictly smaller than n for large n . Consequently, the total running time is $\mathcal{O}(n)$. $\square$

In conclusion, the previous two theorems immediately prove Theorem 6.1, our main result, which we restate here for convenience.

Theorem 6.1 *There exists a randomized algorithm to compute the median that runs in time $\mathcal{O}(n)$ and succeeds with probability $1 - \mathcal{O}(n^{-1/4})$.*

Discussion and Extensions

Theorems 8.1 and 8.2 jointly show that Algorithm 2 achieves $\mathcal{O}(n)$ time with high probability of success. In rare cases where the check fails, the algorithm returns *failure* rather than risking an incorrect answer. One can either re-run the algorithm (the probability of repeated failures being very low) or combine it with a fallback deterministic linear-time median-finding routine such as the median-of-medians approach. The randomized algorithm typically attains lower hidden constants in practice than purely deterministic linear-time solutions, thereby offering an attractive balance of theoretical guarantees and real-world performance.

In the next chapter, we summarize our results and highlight possible directions for future work on randomized selection algorithms.

Conclusion

In this thesis, we presented a randomized selection algorithm for identifying the median of an unsorted array in expected linear time. Our method hinges on sampling a sublinear number of elements, sorting them to obtain pivots, and then “zooming in” on a small region of the array that is guaranteed, with high probability, to contain the true median. The crux of our analysis relied on Chebyshev’s inequality, which provided a principled way to bound the probability that our random pivots deviate significantly from the actual median’s rank in the sorted order.

Summary of Contributions.

- We introduced a randomized median-finding algorithm that leverages a sample of size roughly $n^{3/4}$. Through sorting this sample and selecting pivots near its middle, the algorithm narrows down the search space to a sub-array whose size is no more than $4n^{3/4}$.
- We proved a high-probability bound (Theorem 8.1) on the correctness of the algorithm. Specifically, with probability at least $1 - \mathcal{O}(n^{-1/4})$, our pivots capture the true median, ensuring that the algorithm does not report *failure*.
- We established a near-linear worst-case running time (Theorem 8.2). The cost of sorting the small sample and, conditionally, the sub-array T both remain asymptotically lower than $\mathcal{O}(n)$, leaving the dominant cost to be a single pass over the array to compute element ranks relative to the pivots. Hence, the overall algorithm runs in $\mathcal{O}(n)$ time with high probability.
- Beyond the immediate theoretical guarantees, we discussed how to combine our randomized approach with a deterministic “median-of-medians” fallback to handle rare failure events, ensuring correctness

remains absolute. This hybrid strategy preserves the practical advantages of randomization while delivering a worst-case linear-time bound.

Limitations and Possible Extensions. While our use of Chebyshev’s inequality is conceptually simple and broadly applicable, tighter concentration results (e.g. Chernoff or Hoeffding bounds) could potentially yield smaller failure probabilities or allow an even smaller sample size. However, such refinements might require more specific assumptions about the distribution of the input data or the independence structure of the sampling process.

Another direction for future exploration is to extend the approach to streaming and dynamic settings. In scenarios where the dataset arrives incrementally, maintaining a small, representative sample to approximate the median could be achieved through reservoir sampling or related techniques. Additionally, one might investigate parallel or distributed variants of the algorithm, where the sampling and pivot-selection steps are split across multiple processors or machines.

Bibliography

- [1] Manuel Blum, Robert W Floyd, Vaughan R Pratt, Ronald L Rivest, and Robert E Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.
- [2] Robert Bringhurst. *The Elements of Typographic Style*. Hartley & Marks, 1996.

Dummy Appendix

You can defer lengthy calculations that would otherwise only interrupt the flow of your thesis to an appendix.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

First name(s):

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Signature(s)

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).